

# **Cheat Sheet DevSecOps**

Architecture, Nommage et Commandes (C/C++, Python, Java)

Référentiel CI/CD & Qualité de Code

7 avril 2026

# Table des matières

|          |                                                                                               |          |
|----------|-----------------------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Architecture et Structure des Projets</b>                                                  | <b>2</b> |
| 1.1      | Structure Standard C, C++ et Python . . . . .                                                 | 2        |
| 1.2      | Structure Standard Java (Maven / Gradle) . . . . .                                            | 2        |
| <b>2</b> | <b>Règles de Nommage (Production et Tests)</b>                                                | <b>3</b> |
| <b>3</b> | <b>Dictionnaire des Commandes</b>                                                             | <b>4</b> |
| 3.1      | Dictionnaire des Commandes : C et C++ . . . . .                                               | 4        |
| 3.1.1    | Compilation et Vérification (GCC / Clang) . . . . .                                           | 4        |
| 3.1.2    | Formatage Automatique : <code>clang-format</code> . . . . .                                   | 4        |
| 3.1.3    | Analyse Statique (SAST & Qualité) : <code>cppcheck</code> & <code>flawfinder</code> . . . . . | 4        |
| 3.1.4    | Couverture de Code : <code>gcovr</code> . . . . .                                             | 5        |
| 3.2      | Dictionnaire des Commandes : Python . . . . .                                                 | 6        |
| 3.2.1    | Formatage et Linting : <code>ruff</code> (Remplace Pylint/Black) . . . . .                    | 6        |
| 3.2.2    | Typage Statique : <code>mypy</code> . . . . .                                                 | 6        |
| 3.2.3    | Sécurité SAST : <code>bandit</code> . . . . .                                                 | 6        |
| 3.2.4    | Tests et Couverture : <code>pytest</code> & <code>coverage</code> . . . . .                   | 6        |
| 3.3      | Dictionnaire des Commandes : Java . . . . .                                                   | 7        |
| 3.3.1    | Compilation et Tests : Maven ( <code>mvn</code> ) . . . . .                                   | 7        |
| 3.3.2    | Couverture de Code : JaCoCo . . . . .                                                         | 7        |
| 3.3.3    | Analyse SonarQube et SAST . . . . .                                                           | 7        |
| 3.4      | Résumé du Pipeline CI/CD DevSecOps . . . . .                                                  | 8        |

# Chapitre 1

## Architecture et Structure des Projets

L'organisation des fichiers doit respecter le principe du **Mirroring** (effet miroir). Le dossier des tests doit reproduire exactement l'arborescence du code source.

### 1.1 Structure Standard C, C++ et Python

```
mon-projet/  
|-- src/                # Code source de production  
|   |-- api/  
|   |   \-- auth.py     # ou auth.c / auth.cpp  
|   \-- utils/  
|       \-- math.py  
\-- tests/              # Dossier dedie aux tests (DETECTE PAR LA CI)  
    |-- api/  
    |   \-- test_auth.py  
    \-- utils/  
        \-- test_math.py
```

### 1.2 Structure Standard Java (Maven / Gradle)

Java impose une arborescence très stricte basée sur les conventions Maven.

```
mon-projet-java/  
|-- src/  
|   |-- main/  
|   |   \-- java/  
|   |       \-- com/monentreprise/UserService.java  
|   \-- test/                # Dossier dedie aux tests Java  
|       \-- java/  
|           \-- com/monentreprise/UserServiceTest.java  
\-- pom.xml                  # Fichier de configuration Maven
```

## Chapitre 2

# Règles de Nommage (Production et Tests)

| Langage | Code Source (Variables, Classes)                                         | Fichiers de Tests (Imposé par la CI)                                                               |
|---------|--------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| C / C++ | snake_case pour variables/fonctions.<br>MAJUSCULES pour macros et enums. | test_*.c ou *_test.cpp<br><i>Ex : test_calcul.c</i>                                                |
| Python  | snake_case pour variables/fonctions.<br>PascalCase pour les classes.     | test_*.py ou *_test.py<br><i>Ex : test_auth.py</i>                                                 |
| Java    | camelCase pour variables/méthodes.<br>PascalCase pour les classes.       | *Test.java,                      Test*.java,<br>*TestCase.java<br><i>Ex : UserServiceTest.java</i> |

## Chapitre 3

# Dictionnaire des Commandes

### 3.1 Dictionnaire des Commandes : C et C++

#### 3.1.1 Compilation et Vérification (GCC / Clang)

- `gcc` / `clang` : Compilateurs C.
- `g++` / `clang++` : Compilateurs C++.
- **Options clés :**
  - `-std=c11` ou `-std=c++17` : Force la norme du langage.
  - `-Wall` `-Wextra` : Active tous les avertissements (warnings) majeurs.
  - `-Werror` : Transforme tous les warnings en erreurs (bloque la compilation).
  - `-Wshadow` : Alerte si une variable locale masque une variable globale.
  - `-O2` : Niveau d'optimisation standard pour la production.
  - `-g` : Inclut les symboles de débogage (utile pour Valgrind ou GDB).
  - `-fsanitize=address` : Active ASan (détecte les dépassements de mémoire au runtime).

#### 3.1.2 Formatage Automatique : `clang-format`

Assure un style de code homogène sans débat humain.

- `-i` : Modifie les fichiers directement (in-place). Sans cette option, il affiche juste le résultat dans le terminal.
- `-style=LLVM` (ou GNU, Google) : Applique les règles de formatage spécifiques d'une norme.

#### 3.1.3 Analyse Statique (SAST & Qualité) : `cppcheck` & `flawfinder`

- `cppcheck` : Linter C/C++.
  - `-enable=all` : Active toutes les vérifications (style, perfs, portabilité).
  - `-inconclusive` : Autorise l'outil à remonter des erreurs probables, même s'il n'est pas sûr à 100%.
  - `-xml` : Exporte le résultat au format XML (nécessaire pour SonarQube).
  - `-suppress=missingIncludeSystem` : Ignore les erreurs liées aux entêtes système (ex : `<stdio.h>`).
- `flawfinder` : Scanneur de vulnérabilités de sécurité (C/C++).
  - `-singleline` : Affiche les vulnérabilités sur une seule ligne (plus facile à lire dans les logs CI).

### 3.1.4 Couverture de Code : `gcovr`

Extrait les données de couverture générées lors de l'exécution des tests.

- `-xml-pretty` : Génère un fichier XML lisible (format Cobertura pour SonarQube).
- `-print-summary` : Affiche un résumé rapide dans le terminal de la CI.
- `-o coverage.xml` : Définit le nom du fichier de sortie.

## 3.2 Dictionnaire des Commandes : Python

### 3.2.1 Formatage et Linting : ruff (Remplace Pylint/Black)

Ruff est un linter et formateur ultra-rapide écrit en Rust, utilisé dans notre pipeline moderne.

- `python -m ruff format .` : Formate automatiquement le code.
- `python -m ruff format -check .` : Vérifie si le code est bien formaté (Renvoie une erreur "exit 1" si ce n'est pas le cas, utile en CI).
- `python -m ruff check .` : Lance l'analyse statique (détecte les variables inutilisées, les imports morts, etc.).

### 3.2.2 Typage Statique : mypy

- `python -m mypy .` : Vérifie les Type Hints (`x: int, -> str`).
- `-strict` : Option facultative mais recommandée. Force le typage de **toutes** les fonctions sans exception.

### 3.2.3 Sécurité SAST : bandit

Analyse le code source Python pour trouver des failles de sécurité.

- `-r .` : Scanne le dossier courant récursivement.
- `-f xml -o bandit-report.xml` : Spécifie le format de sortie (`xml`) et le fichier cible pour SonarQube.

### 3.2.4 Tests et Couverture : pytest & coverage

- `python -m coverage run -m pytest` : Lance les tests `pytest` tout en enregistrant quelles lignes de code ont été exécutées.
- `-q` (pour `pytest`) : Mode "quiet" (silencieux), réduit la verbosité dans la CI.
- `-junitxml=pytest-report.xml` : Exporte le résultat (succès/échecs des tests) en format standardisé XML.
- `python -m coverage xml` : Transforme les données de couverture enregistrées en un fichier `coverage.xml` lisible par SonarQube.

## 3.3 Dictionnaire des Commandes : Java

### 3.3.1 Compilation et Tests : Maven (`mvn`)

Maven est l'outil de gestion de build standard.

- `mvn clean` : Supprime le dossier `target/` (les anciens builds) pour repartir de zéro.
- `mvn test` : Compile le projet et lance les tests unitaires via le plugin Surefire (qui cherche les fichiers `*Test.java`).
- `-fn` (Fail Never) : Option cruciale en CI non-bloquante. Même si un test échoue, Maven ne retourne pas d'erreur fatale, permettant au pipeline de continuer et d'envoyer le rapport à SonarQube.

### 3.3.2 Couverture de Code : JaCoCo

- `mvn jacoco:report` : Génère le rapport de couverture XML dans le dossier `target/site/jacoco/jacoco.xml`. Doit être lancé **après** l'exécution des tests.

### 3.3.3 Analyse SonarQube et SAST

En Java, SonarQube fait souvent l'analyse statique lui-même, mais il peut être complété.

- `mvn sonar:sonar` : Envoie le code, les rapports de test (Surefire) et les rapports de couverture (JaCoCo) vers le serveur SonarQube.
- `-Dsonar.projectKey=...` : Définit l'identifiant unique du projet sur le serveur Sonar.
- `-Dsonar.host.url=...` : URL du serveur SonarQube (ex : `http://localhost:9000`).
- `-Dsonar.token=...` : Jeton d'authentification pour se connecter au serveur.

### 3.4 Résumé du Pipeline CI/CD DevSecOps

La logique appliquée à tous ces outils dans notre GitHub Actions repose sur deux règles de base :

1. **Vérification bloquante du Format et du Style** : Si le code est mal nommé (ex : `test_*.py`), mal indenté (`clang-format`, `Ruff`), la CI s'arrête en erreur (`exit 1`). Le développeur doit corriger localement et pousser de nouveau.
2. **Vérification non-bloquante des Tests (Best Effort)** : Si aucun test n'est trouvé, la CI crée un rapport vide factice (`touch coverage.xml`) pour ne pas faire planter SonarQube. L'objectif est de mesurer la qualité continue, pas de bloquer le projet s'il n'y a pas encore de tests.